or break EXPRESSION.METHOD, both of which set a breakpoint at the beginning of the given method. All of those options can add an if EXPRESSION such that the breakpoint only stops the code if the expression is true. You also have the suffixes pre: COMMAND and do: COMMAND, which run the command at the breakpoint—the difference being that pre stops the debugger, and do runs the command and keeps going.

Typing b by itself with no other arguments will give you a numbered list of breakpoints, which is useful because del NUM allows you to delete a specific breakpoint, and just del with no numbers deletes them all.

A reasonable thing to do in a debugger is look for errors, and the Ruby debugger allows you to catch ERROR as in catch ArgumentError, which stops execution when an exception of that type is raised, at which point you can continue to navigate or query the system using the methods in the next section. The catch command also takes the if, pre, and do suffixes just like break.

You can also cause the debugger to stop when a value is changed—in kind-of a limited way. The watch command takes an instance variable name, as in watch @name, and stops execution if that instance variable of the object in the current scope changes. The official docs do point out that this is, and we quote, "super slow." The watch command also takes if, pre, and do suffixes.

Querying the System

When you are stopped via a breakpoint is a great time to look around and try to find out what the code is doing.

Most generally, you can type eval EXPRESSION and evaluate an arbitrary expression in the context of the current set of active variables. You can even use this to change the value of variables, which means you can easily break things if you want, or you can explicitly set up edge cases that are hard to place. You can also just p EXPRESSION or pp EXPRESSION that prints or pretty-prints the result of the expression. And you can type irb, which will take you into an irb session with the current set of variable values.

You can specifically see the current call stack with bt or backtrace. If you put a number after the command, the debugger will only show that number of lines; if you put a regular expression after the command, it will only show you lines that match the expression. The i command will show you all the values active at the current point, and i i and i l will show instance values and local values, respectively.

The debugger has the concept of a "tracer" where you can get a message from the debugger when running code triggers an event. The command trace call will show you all method calls (and I mean *all* method calls). You can add a regular expression on the end to filter the calls you see. You can use trace exception to see what exceptions get raised, and trace line to track lines of code as the script bounces along.

There's more to the debugger, you can see a full list of commands at https://github.com/ruby/debug.